

Reviewer Pack — Hodge Index vs Resolution Proof Tree Diameter

Entry 1e5dbd3e6410 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 04:37:13 UTC

Hodge Index vs Resolution Proof Tree Diameter

Entry ID: 1e5dbd3e6410 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 04:37:13 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): Complexity Theory (Resolution Proof Complexity)

Statement:

[‘For every satisfiable CNF F with m clauses and n variables, the Hodge index $h(F)$ of the associated Hodge bundle on the moduli space of complex curves is upper bounded by a constant multiple of the diameter d_F of its resolution proof tree.’, ‘Equivalently, if we denote the number of leaves in the resolution proof tree as l_F , then $h(F) \leq c * \log_2(l_F)$, where c is an absolute constant.’]

Rationale (proposer’s reasoning):

[‘Hodge theory provides a rich algebraic-geometric framework that may reveal structural insights into the complexity of resolution proofs. The Hodge index could serve as a potential invariant that correlates with the complexity of computational problems, especially those related to proof complexity.’, ‘The connection between the Hodge index and the diameter of the resolution proof tree is motivated by the geometric nature of Hodge theory and its potential to capture the topological aspects of resolution proofs.’]

Taxonomy category: HODGE_INTEGRAL (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 264942c998ac40ce

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For each CNF F with m clauses and n variables, if the Hodge index $h(F)$ is less than or equal to a constant multiple c of the log base 2 of the number of leaves l_F in its resolution proof tree ($h(F) \leq c * \log_2(l_F)$), then the conjecture is supported. If any seed produces a case where $h(F) > c * \log_2(l_F)$, and this condition is consistent across all seeds, the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	0.95	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Hodge index" AND "resolution proof tree diameter" - "algebraic geometry" AND "complexity theory" AND "CNF satisfiability" - "Hodge bundle" AND "resolution proof complexity" AND "diameter"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
import itertools

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n, m):
        clauses = []
        for _ in range(m):
            clause = [random.randint(1, n), -random.randint(1, n)]
            clauses.append(clause)
        return clauses

    def resolution_tree(cnf):
        # Simplified resolution tree construction
        leaves = set(range(1, n + 1))
        tree = {i: [] for i in range(1, n + 1)}
        for clause in cnf:
            if len(clause) == 2 and abs(clause[0]) != abs(clause[1]):
                leaves.remove(abs(clause[0]))
                leaves.remove(abs(clause[1]))
                tree[abs(clause[0])].append(abs(clause[1]))
                tree[abs(clause[1])].append(abs(clause[0]))
        return tree, len(leaves)

    def hodge_index(n):
        # Simplified Hodge index calculation
        return n * (n - 1) // 2
```

```

n = random.randint(5, 40)
m = random.randint(2 * n, 3 * n)
cnf = generate_cnf(n, m)
tree, l_F = resolution_tree(cnf)

h_F = hodge_index(n)
c = 1.0 # Placeholder constant
log_l_F = math.log2(l_F) if l_F > 0 else float('inf')

metric_value = h_F
conjecture_holds = h_F <= c * log_l_F
counterexample = "" if conjecture_holds else f"Counterexample: n={n}, m={m}, h(F)={h_F}, c*log

return {
    "metric_name": "Hodge Index vs Resolution Proof Tree Diameter",
    "metric_value": metric_value,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else list(range(2, 103, 4))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6f96fd47.py", line 69, in <module>

```

    result = run_trial(seed)
    ^^^^^^^^^^^^^^^^^

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6f96fd47.py", line 46, in run_trial

```

    tree, l_F = resolution_tree(cnf)

```

```

^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6f96fd47.py", line 33, in resolution_tr
    leaves.remove(abs(phrase[0]))
KeyError: 33

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution, which prevents us from verifying the conjecture's validity. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14160
2	preregistration	ollama_remote	glm4:latest	0	0	6179
3	novelty	ollama_remote	glm4:latest	0	0	4775
4	novelty	ollama_remote	glm4:latest	0	0	5229
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17322
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8344
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8548
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8783
9	judge	ollama_remote	glm4:latest	0	0	8992

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 82333 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/1e5dbd3e6410.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1e5dbd3e6410.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/1e5dbd3e6410.tar.gz` (if generated)